

Explaining Addition

The CTC proposes to explain cognitive capacities by appeal to representation and computation in exactly the way that arithmetical capacities of calculators (such as addition) are standardly explained by appeal to representation and computation. The main explanatory strength of the CTC is that it proposes to explain cognition in terms of antecedently understood notions of representation and computation— notions the CTC takes to be unproblematic in a variety of familiar noncognitive contexts such as calculating and elementary computer programming. We would do well to begin, then, by reviewing the “received” explanation of addition in adding machines.

To add is to be described by the plus function, $+(⟨m, n⟩) = s$. Hence, to explain what makes a system an adder is to explain the fact that the system is described by $+$. But $+$ is a function whose arguments and values are numbers; and whatever numbers are, they are not states or processes or events in any physical system. How, then, can a physical system be described by $+$? How can a physical system traffic in numbers and hence add? The answer, of course, is that *numerals*—i.e., representations of numbers—can be states of a physical system, even if the numbers themselves cannot. A physical system adds by trafficking in numerals and hence, indirectly, in the numbers those numerals represent.

The input to a typical adding machine is a sequence of button pressings: $\langle C, M, +, N, = \rangle$, i.e., $\langle \text{clear, first addend, plus, second addend, equals} \rangle$. The output is a display state, D , which is a numeral representing the sum of the two addends. We may think of the button-pressing sequences as arguments to a function g that gives display states as values. An adding machine *satisfies* g ; that is, the arguments and values of g are literally states of the physical system.¹ Addition, as was remarked above, relates numbers, not physical states of some machine, so a physical system cannot literally satisfy the plus function. What an adding machine does is *instantiate* the plus function. It *instantiates* addition by *satisfying* the function g whose arguments and values represent the arguments and values of the addition function, or in other words, have those arguments and values as interpretations.

Putting all this together, we have that something is an adding machine because

1. we can interpret button pressings (or the internal states they cause) as numbers,²
2. we can interpret displays (or the internal states that cause them) as numbers.
3. the device causally associates sequences of button pressings with displays (i.e., it satisfies a button-pressing-to-display function),

and, given all this,

4. if a token of the button-pressing events interpreted as n and m were to occur, then a token display event interpreted as $n+m$ would normally occur as a consequence. The device instantiates the addition function by satisfying the function mentioned in step 3, for that function is interpretable as the addition function.

To get a physical device to add, you have to get it to satisfy a function interpretable as addition. And that means you have to design it so that getting it to represent a pair of addends causes it to represent their sum.

The following is a useful picture of this whole conception of adding machines. I call it the Tower Bridge picture because it reminds me of London's Tower Bridge.

$$\begin{array}{ccc}
 +: I \langle \langle C, N, +, M, = \rangle \rangle = \langle n, m \rangle & \longrightarrow & I(D) = n + m \\
 \uparrow I & & \uparrow I \\
 g: \langle C, N, +, M, = \rangle = (\text{computation}) & \Longrightarrow D & \Longrightarrow
 \end{array}$$

The top span pictures the function instantiated: $+$, in our present case. It takes a pair of numbers onto their sum. The bottom span corresponds to the function satisfied (called simply g). It takes a

quintuple of button pressings onto a display. The vertical arrows correspond to interpretation: $I \langle \langle C, N, +, M, = \rangle \rangle$ is the interpretation of $\langle \langle C, N, +, M, = \rangle \rangle$, namely $\langle n, m \rangle$, the pair of numbers represented by N and M . $I(D)$ is the interpretation of the display, which will be the sum of n and m if the thing works right. Under interpretation, the bottom span is revealed as an instantiation of the top span; computation is revealed as addition.

Computation

We have reduced the problem of explaining addition to the problem of explaining why the machine satisfies g , the function that instantiates addition. In standard adding machines, the values of g are *computed* from the corresponding arguments; adding machines add by computing the appropriate representations.

Of course, functions need not be computed to be satisfied. Set mousetraps satisfy a function from trippings to snappings without computing it, and physical objects of all kinds satisfy mechanical functions without computing them. The planets stay in their orbits without computing them. Missiles, on the other hand, compute their trajectories (sometimes), and, in general, complex systems often satisfy functions by computing them. Humans, in particular, routinely satisfy functions by computing them. This is how recipes and instruction manuals enable you to satisfy functions you don't know how to satisfy directly. The recipe for hollandaise sauce, for example, specifies such things as eggs and double boilers as inputs and hollandaise sauce as output. When you execute such a recipe, you satisfy a function having the recipe inputs as arguments and the hollandaise (the recipe output) as value. Recipes analyze such functions into simple functions you know how to satisfy directly—e.g., add three eggs to the contents of the bowl.³

To compute a function g is to execute a program that gives o as its output on input i just in case $g(i) = o$. Computing reduces to program execution, so our problem reduces to explaining what it is to execute a program.

The obvious strategy is to exploit the idea that program execu-

tion involves *steps*, and to treat each elementary step as a function that the executing system simply *satisfies*. To execute a program is to satisfy the steps.

But what guarantees that the steps are executed—i.e., satisfied—“in the right order”? Program execution is surely disciplined step satisfaction. Where does the discipline come in? The discipline takes care of itself. Functions satisfied by d specify causal connections between events in d , so if d satisfies f and g and if the current value of f is an argument for g , then an execution of the f step will produce an execution of the g step. Program execution reduces to step satisfaction.⁴

You won't go far wrong if you think of it this way. Imagine the program expressed as a flow chart. Each box in the chart represents a step. To execute a step is to satisfy its characteristic function, i.e., the function specified by the input/output properties of the box. If you think of the arrows between the boxes as causal arrows, the result is a causal network with steps (i.e., functions to be satisfied) at the nodes. A system executes the program if that causal network gives the (or a) causal structure of the system.⁵

The Role of Representation in This Explanation

The arguments and values of the button-pressing-to-display function that play the role of representations—i.e., the button-pressing sequences and the display states themselves—need not be symbols with any use or meaning outside the system that computes them. What matters is only that the system satisfy a function g that instantiates $+$. That is, there must exist an interpretation I such that

$$g(x) = y \quad \text{iff} \quad +(I(x)) = I(y).$$

It is, in short, simply the fact that g is isomorphic to $+$ that makes the arguments and values of g representations of numbers for a system that satisfies g . It is sufficient for the arguments and values of g 's being representations of addends and sums that there exist an interpretation mapping g onto $+$: The arguments

and values of g 's being representations of numbers is *constituted* by the fact that g instantiates $+$; representation is just a name for the relation induced by the interpretation mapping between the elements of g and the elements of $+$.

That there is no "further fact"⁶ to representation in this case beyond g 's instantiating $+$ is obscured by the use of symbols with a meaning that is independent of their use in the system. One is tempted to suppose that the display state that looks like "5" would mean five regardless of whether g actually instantiates $+$. And so it would—to us. But its meaning in the system might be something quite different. To see this, we simply have to imagine that the buttons are mislabeled and that the display wires are crossed. In such a case we would have to discover what represented what, and this we would do by establishing a mapping, I , between g and $+$ such that

$$g(x) = y \quad \text{iff} \quad +(I(x)) = I(y).$$

Indeed, it suffices to notice that it *makes sense* to suppose that the buttons *could* be mislabeled, or that the display wires *could* be scrambled, for this would make no sense if the meanings of input and output states were determined by the conventional meanings of those states considered as symbol tokens. We say that a display is scrambled because the conventional meanings of the display don't match the meanings that they have in the system, and this evidently requires that displays have a meaning in the system that is independent of their conventional meanings.

We are now in a position to see that it is somewhat misleading to speak of the explanatory role of representation in this context. The explanatory burden is carried by the fact of simulation and the correlative concept of interpretation. There is a sense in which an adding machine adds because it represents numbers, but there is a more important sense in which it represents numbers because it adds: We can speak of it as representing numbers only because it stimulates $+$ under some interpretation.

From an explanatory point of view, what interpretation provides is a link between mere state crunching (button-pressing-to-

display transitions) and *addition*: Under interpretation, the state transitions of the system are revealed as adding. You can get a feel for the explanatory role of interpretation by imagining that the device is an archaeological discovery. Perhaps you know from documents that the thing is an adding machine; however, looking at it, you have no idea, at first, how to give inputs or read outputs. You can “make it go,” let’s say, by turning a handle, but this is just more or less random manipulation until you find an interpretation that reveals the state transitions of the device as sum calculations and hence establishes the representational status and contents of those states. To know how to use it is to know how to see it as simulating +, and that means knowing how to “make it go” and how to interpret it.

The concept of representation that is operative here can be brought more clearly into focus by considering again Galileo’s ingenious use of geometrical figures to represent mechanical magnitudes (figure 8.1). The crucial point is that, given Galileo’s interpretation of the lines and volumes, the laws of Euclidean geometry discipline those representations in a way that mirrors the way the laws of mechanics discipline the represented magnitudes: The geometrical discipline mirrors the natural discipline of the domain. That is, geometrical relationships among the symbols have counterparts in the natural relations among mechanical variables⁷ in such a way that computational transformations on the symbols track⁸ natural transformations of the system. That is what makes it correct to say that the symbols—lines and volumes—*represent* times, velocities, and distances.

Of course, this is what Galileo *intended* them to represent; that is the interpretation he stipulated. But it is one thing to intend to represent something, another to succeed. Galileo’s figures *actually do* represent mechanical variables because the computational discipline *actually does* track the natural one. This is the natural discipline we have in mind when we say that the system behaves according to natural law. I call Galileo’s interpretation a proper interpretation because under that interpretation the natural system and the geometrical system that represents it march in step: The geometrical system *simulates* the natural one.⁹ A proper